

Chapter 2 - Components, Ports, PCells & Mini-PDK

GDSFactory / Silicon Photonics Design Automation Cheat Sheet

Purpose: a compact reference for the concepts learned before the Chapter 2 mini-project.

Big Picture

```
technology definition
  |
  v
cross-section
  |
  v
parameterized PCell
  |
  v
ports / references / connections
  |
  v
mini-PDK
  |
  v
automated tests
  |
  v
GDS / OASIS
```

Topic	What to remember	Simple example
2.1 Ports	Ports describe how components connect: name, center, width, orientation, layer, type.	<code>wg.ports["o1"]</code>
2.2 References	A Component is a definition; a Reference is a placed instance that can move, rotate, mirror, and connect.	<code>wg_ref = c << wg</code>
2.3 Cross-sections	A cross-section describes the physical waveguide technology.	<code>gf.cross_section.strip(width=0.5)</code>
2.4 Tapers	Gradually transition between different waveguide widths.	<code>taper(width1=0.45, width2=1.0)</code>
2.5 MMI / Coupler	Reusable multi-port passive components.	<code>mmi1x2(), coupler()</code>
2.6 PCells	<code>@gf.cell</code> creates reusable parameterized GDSFactory cells.	<code>@gf.cell def taper_amjuk(...)</code>
2.7 Mini-PDK	Centralize layers, cross-sections, and approved component generators.	<code>WG=(1,0) -> strip_xs() -> pdk_straight()</code>
2.8 Tests	Automatically verify ports, dimensions, layers, and deterministic behavior.	<code>assert len(c.ports) == 2</code>
2.9 GDS/OASIS	Two layout-file formats for hierarchical mask data.	<code>.gds, .oas</code>

1. Component vs Reference

This is one of the most important mental models in GDSFactory.

```
wg = gf.components.straight(length=20)
```

wg is a **Component**: the cell definition.

```
wg_ref = c << wg
```

wg_ref is a **Reference**: a placed instance of that Component.

```
wg_ref.move((20, 10))
wg_ref.rotate(90)
```

Normally, you transform the reference, not the component definition.

2. Ports

```
wg_ref.ports["o1"]
```

Important port properties: **name**, **center**, **width**, **orientation**, **layer**, **port_type**.

```
o1 ----- o2
```

horizontal straight:

```
o1 orientation ~ 180 deg
```

```
o2 orientation ~ 0 deg
```

```
ref1.connect("o1", ref2.ports["o2"])
```

`connect()` moves/rotates `ref1` so the selected ports coincide correctly. It does **not** create a routing waveguide between distant components.

3. Cross-section

Path = where light travels

CrossSection = what waveguide technology travels along that path

Component = final geometry

```
xs = gf.cross_section.strip(
    width=0.5,
    layer="WG",
)
```

```
wg = gf.components.straight(
    length=20,
    cross_section=xs,
)
```

A cross-section may contain width, layer, offset, multiple sections, and bend-radius information.

A cross-section is **not** a Component, so a cross-section factory normally does not use `@gf.cell`.

4. Taper

```
taper = gf.components.taper(
    length=15,
    width1=0.45,
    width2=1.0,
)
```

```
0.45 um          1.0 um
-----\_____/-----
```

A gradual transition reduces abrupt mode mismatch and unwanted radiation/mode coupling.

```
0.45 straight
  |
  v
  taper
  |
  v
1.0 straight
```

5. MMI

```
mmi = gf.components.mmi1x2()

      o2
      /
o1 -> [ MMI ]
      \
      o3
```

The broad central region is multimode. Forward use: **1 input -> 2 outputs**. Reverse use: **2 inputs -> 1 output**. Physical mirroring is not automatically required just to reverse the optical role.

6. Directional Coupler

```
dc = gf.components.coupler(
    gap=0.2,
    length=20,
)

WG1 =====
    evanescent
    coupling
WG2 =====
```

Main geometric parameters: **gap** and **interaction length**. Coupling is not simply "longer = always more coupling."

Idealized relation: $P_{\text{cross}} = \sin^2(\kappa * L)$. Power can oscillate between the two guides.

```
gaps = [0.15, 0.20, 0.25]
lengths = [5, 10, 15, 20, 25]

for i, gap in enumerate(gaps):
    for j, length in enumerate(lengths):
        dc = gf.components.coupler(
            gap=gap,
            length=length,
        )
```

This nested-loop sweep is an early example of design automation.

7. Your Own PCell

```
@gf.cell
def taper_amjuk(
    width1=0.45,
    width2=1.0,
    length1=10,
    length2=20,
    taper_length=15,
):
    ...
    return c

function parameters
    |
    v
generate geometry
    |
    v
return Component

device = taper_amjuk(
    width1=0.5,
    width2=1.2,
    taper_length=25,
)
```

Custom parent-level ports make the entire device behave as one reusable component:

```
c.add_port("A_in", port=wg1_ref.ports["o1"])
```

```
c.add_port("A_out", port=wg2_ref.ports["o2"])
```

8. Validation

```
if width1 <= 0:
    raise ValueError("width1 must be positive")
```

A reusable component should reject physically meaningless inputs instead of silently producing bad geometry. This becomes especially important when automation generates many devices.

9. Deterministic Naming and Caching

```
taper_amjuk_w0p45_w1_L10_L20_TL15
taper_amjuk_w0p45_w1_L10_L20_TL15
taper_amjuk_w0p45_w1_L10_L20_TL25
```

```
same function + same parameters
-> same deterministic cell name
```

```
different parameter
-> different cell name
```

Caching means that GDSFactory can reuse an already-generated cell definition for the same `@gf.cell` + parameter set instead of rebuilding it unnecessarily. Deterministic naming and caching are related, but they are not exactly the same thing.

10. Organizing Components into Modules

```
components/
|-- tapers.py
|-- waveguides.py
`-- couplers.py

# waveguides.py

@gf.cell
def custom_waveguid(length=20, width=0.5):
    ...

from waveguides import custom_waveguid
```

Basic import mental model: `file.py` -> function inside file. Example: `from tapers import taper_amjuk` means "import `taper_amjuk()` from `tapers.py`."

11. Mini-PDK

```
pdk/
|-- layers.py
|-- cross_sections.py
`-- components.py
```

layers.py

```
WG = (1, 0)
SLAB = (2, 0)
METAL = (10, 0)
TEXT = (100, 0)

(1, 0)
 ^  ^
 |  `-- datatype
`----- GDS layer number
```

GDS layer numbers are identifiers, not physical vertical positions in the stack. Their physical meaning comes from the PDK/process definition.

cross_sections.py

```
def strip_xs(width=0.5):
    return gf.cross_section.strip(
        width=width,
        layer=WG,
    )
```

Now the waveguide technology comes from one centralized definition.

components.py

```
@gf.cell
def pdk_straight(length=20, width=0.5):
    c = gf.Component()

    wg = gf.components.straight(
        length=length,
        cross_section=strip_xs(width=width),
    )

    ...
    return c
```

```
layers.py
WG = (1,0)
    |
    v
cross_sections.py
strip_xs()
    |
    v
components.py
pdk_straight()
    |
    v
Component
    |
    v
GDS / OASIS
```

This is the heart of the mini-PDK idea: technology definitions are centralized instead of being hard-coded throughout individual scripts.

12. @gf.cell vs PDK

```
@gf.cell
= mechanism for making a reusable parameterized Component
```

```
PDK
= framework containing technology information and component definitions
```

```
PDK
|-- layers
|-- cross-sections
`-- @gf.cell PCells
```

A PDK can contain many `@gf.cell` functions. They are not competing concepts.

13. Automated Tests with pytest

```
tests/
`-- test_components.py

def test_pdk_straight_has_two_ports():
    c = pdk_straight(length=20, width=0.5)
    assert len(c.ports) == 2
```

`assert` means: this condition must be true.

```
python -m pytest tests/test_components.py -q
```

Examples of the tests you wrote:

```
assert len(c.ports) == 2

assert "o1" in c.ports
assert "o2" in c.ports

with pytest.raises(ValueError):
    pdk_straight(width=-0.5)

assert a.name == b.name

assert tuple(c.ports["o1"].layer) == WG
```

14. GDSII vs OASIS

```
device.write_gds("device.gds")
device.write("device.oas")
```

In your example, the generated files were:

```
GDS  = 2258 bytes
OAS  = 1495 bytes

GDSII
-> older
-> extremely common
-> .gds

OASIS
-> newer
-> often more compact
-> .oas
```

Both are layout formats and can be opened directly in KLayout. OASIS is often more efficient for large, repetitive, hierarchical designs.

The One Mental Picture to Keep

```
LAYERS
WG = (1,0)
|
v
CROSS-SECTION
strip_xs(width=0.5)
|
v
PCELL
@gf.cell
pdk_straight(...)
|
v
COMPONENT
ports + geometry
|
v
REFERENCE
c << component
|
v
CIRCUIT / HIERARCHY
connect / place
|
v
TEST
pytest
|
v
EXPORT
```

Chapter 2 in one sentence: You moved from creating isolated GDSFactory objects to building reusable, parameterized, PDK-aware photonic components that can be tested automatically and exported as hierarchical layout data.

End of cheat sheet - ready for the Chapter 2 mini-project.